Ordenamiento, en la práctica

Adaptación a C#

Todo algoritmo de este libro hasta ahora se ha medido asintóticamente — por cómo crece el coste con el tamaño de la entrada, ignorando las constantes. Ese es el modelo correcto para el análisis teórico y el modelo equivocado para el código de producción. Para cuando llamas a `sorted()` sobre una lista de mil elementos, las constantes importan más que el orden asintótico, y el orden asintótico es una restricción, no una receta.

Este capítulo trata de lo que hacen las implementaciones reales de ordenamiento, y es una reivindicación silenciosa de los cinco capítulos anteriores. Los algoritmos usados en producción — introsort en `std::sort` de C++, Timsort en `sorted()` de Python y `Arrays.sort` de Java y `slice::sort` de Rust — no son invenciones nuevas. Son composiciones de los algoritmos que ya tienes. Ordenamiento por inserción para subarreglos pequeños. Quicksort para el divide recursivo. Merge sort para comportamiento estable y predecible. El arte del ordenamiento práctico es saber qué movimiento aplica dónde, y coserlos de modo que cada uno opere solo sobre las entradas para las que es óptimo.

Dos algoritmos de ordenamiento de estilo producción — un introsort simplificado y un Timsort simplificado — muestran cómo las técnicas de los capítulos 3 a 5 se componen para rendimiento del mundo real. *Un algoritmo de ordenamiento práctico es una composición, no una elección.* No eliges merge sort o quicksort o ordenamiento por inserción. Eliges qué algoritmo corre sobre qué parte de la entrada, y dejas que cada uno haga lo que mejor sabe hacer.

1. Por qué el ordenamiento por inserción se gana su lugar

Asintóticamente, el ordenamiento por inserción es $O(n^2)$ y merge sort es $O(n \log n)$. Para n grande, merge sort gana. Para n pequeño, la comparación asintótica te miente.

La razón es el factor constante. Merge sort asigna arreglos auxiliares, recurre, y hace contabilidad de índices; cada una de esas operaciones cuesta tiempo real por elemento. El ordenamiento por inserción, sobre una lista de 8 elementos que ya está casi ordenada, corre un bucle apretado de comparaciones y un par de intercambios — sin recursión, sin asignación, casi nada más que recorridos lineales por memoria caliente en caché. Sobre un subarreglo de 16 elementos, el ordenamiento por inserción vence a merge sort y a quicksort por un factor de 2 a 4 en tiempo de reloj real, a pesar de ser asintóticamente más lento.

El punto de cruce exacto varía por lenguaje, hardware, e incluso por los datos que se ordenan, pero un umbral de corte en algún lugar entre 8 y 32 es el punto dulce convencional. El Timsort de CPython usa 32. El `std::sort` de C++ típicamente usa 16. La elección es empírica: cada implementación mide la suya.

La receta práctica se escribe sola: cuando la recursión llega a un subarreglo pequeño, deja de recurrir y llama directamente al ordenamiento por inserción. El orden asintótico no cambia — las entradas grandes siguen necesitando un algoritmo $O(n \log n)$ — pero el factor constante mejora dramáticamente porque cada hoja del árbol de recursión es ahora un bucle lineal eficiente en lugar de una llamada recursiva ineficiente.

```
/// <summary>
/// Ordenamiento por insercion in situ sobre items[lo..hi).
/// Bucle apretado, sin recursion.
/// </summary>
```

```
private static void InsertionSortRange<T>(
    IList<T> items, int lo, int hi, Comparison<T> order)
{
    for (int i = lo + 1; i < hi; i++)
    {
        int j = i;
        while (j > lo && order(items[j], items[j - 1]) < 0)
        {
            (items[j], items[j - 1]) = (items[j - 1], items[j]);
            j--;
        }
    }
}
```

Listing 1: Ordenamiento por inserción sobre un rango

Este es el mismo ordenamiento por inserción del capítulo 3, parametrizado para trabajar sobre un subrango en lugar de todo el arreglo — que es lo que los llamadores recursivos necesitan. Es el bloque de construcción para todo lo demás en este capítulo.

2. Introsort: quicksort con redes de seguridad

David Musser introdujo introsort en 1997 para resolver un problema real de quicksort: su peor caso $O(n^2)$, que la selección aleatoria de pivote hace improbable pero no imposible. El "intro" significa *introspectivo*: introsort ejecuta quicksort normalmente, vigila su propia profundidad de recursión, y salta a un algoritmo distinto si la profundidad crece más allá de un umbral sensato — la señal de que las elecciones de pivote han sido desafortunadas y el algoritmo está degenerando hacia comportamiento cuadrático.

El algoritmo completo de Musser tiene tres piezas:

1. **Quicksort** como el algoritmo principal — recursión de arriba hacia abajo, partición alrededor de un pivote, recurre en cada lado.
2. **Ordenamiento por inserción** para subarreglos pequeños (el corte de esquina de la sección anterior).
3. **Heapsort** como respaldo cuando la profundidad de recursión excede $2\log_2 n$ — una garantía contra $O(n^2)$.

La tercera pieza necesita heapsort, que aún no he introducido (aterriza en la Parte III, donde los montículos reciben su tratamiento apropiado). Para el introsort simplificado de este capítulo, me saltaré el respaldo de heapsort e implementaré solo las dos primeras piezas. El resultado es "introsort sin la red de seguridad" — bien para entradas que no están construidas adversarialmente, que es la vasta mayoría de las entradas del mundo real. El límite de profundidad y el respaldo de heapsort son una adición de un párrafo una vez que tienes un montículo, y añadiré ese puntero en las notas al final.

```
/// <summary>
/// Introsort-lite: quicksort con corte a ordenamiento por insercion
/// para subarreglos pequenos.
/// El introsort original de Musser tambien rastrea la profundidad de
/// recursion y cae a heapsort mas alla de 2*log2(n) niveles; esa pieza
/// necesita monticulos (Parte III) y se omite aqui.
/// </summary>
public static void IntroSort<T>(
    IList<T> items, Comparison<T> order = null,
```

```
    int insertionCutoff = 16, Random rng = null)
{
    order ??= Comparer<T>.Default.Compare;
    rng ??= Random.Shared;
    IntroSortRange(items, 0, items.Count, order, insertionCutoff, rng);
}

private static void IntroSortRange<T>(
    IList<T> items, int lo, int hi,
    Comparison<T> order, int cutoff, Random rng)
{
    if (hi - lo <= cutoff)
    {
        InsertionSortRange(items, lo, hi, order);
        return;
    }
    int pivotIdx = rng.Next(lo, hi);
    (items[pivotIdx], items[hi - 1]) = (items[hi - 1], items[pivotIdx]);
    int p = QuickSortImpl.Partition(items, lo, hi, order);
    IntroSortRange(items, lo, p, order, cutoff, rng);
    IntroSortRange(items, p + 1, hi, order, cutoff, rng);
}
```

Listing 2: Introsort-lite

La estructura es exactamente el quicksort del capítulo 4 con dos cambios: un pivote aleatorio (ya que esto está pensado para ser de grado producción), y un corte a ordenamiento por inserción cuando el subarreglo es pequeño.

```
var items = new[] { 3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5, 8, 9, 7, 9, 3, 2,
    3, 8, 4 };
PracticeImpl.IntroSort(items);
Console.WriteLine(string.Join(", ", items));
// ordenado in situ

// Entrada aleatoria grande      el caso realista
var rng = new Random(0);
var big = new int[10000];
for (int i = 0; i < big.Length; i++) big[i] = rng.Next(0, 1001);
PracticeImpl.IntroSort(big, null, 16, rng);
Console.WriteLine($"{string.Join(", ", big[..10])} ... {string.Join(", "
    , big[^10..])}");
```

```
1, 1, 2, 2, 3, 3, 3, 3, 4, 4, 5, 5, 5, 6, 7, 8, 8, 9, 9, 9
0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ... 999, 999, 999, 1000, 1000, 1000, 1000, 1000, 1000, 1000
```

El `std::sort` de C++ ha usado un introsort completo desde alrededor de 1997, y es por eso que los programadores de C++ pueden llamarlo sobre cualquier contenedor sin preocuparse por comportamiento $O(n^2)$ — el límite de profundidad atrapa los casos patológicos y cambia a heapsort, que está garantizado $O(n \log n)$. La composición — quicksort + ordenamiento por inserción + heapsort — cubre todo régimen de entrada: grande recibe quicksort, pequeño recibe ordenamiento por inserción, patológico recibe heapsort.

3. Timsort: merge sort que ve las rachas

Tim Peters escribió Timsort en 2002 para `list.sort()` de Python, después de notar dos hechos estructurales sobre las entradas del mundo real que el argumento de la cota inferior

basada en comparaciones del capítulo 4 ignoraba:

1. **Los datos reales tienen rachas.** Los archivos llegan ordenados por marca de tiempo; los registros de base de datos llegan ordenados por clave primaria; las listas de cara al usuario llegan parcialmente ordenadas porque los usuarios las editan incrementalmente. Una entrada genuinamente aleatoria es el caso raro; el caso común tiene largas subsecuencias ascendentes (o descendentes, que inviertes) ya en su lugar.

2. **El merge sort de abajo hacia arriba evita la sobrecarga de recursión.** En lugar de partir recursivamente hasta llegar a elementos individuales y luego fusionar de vuelta hacia arriba, empieza tratando cada elemento (o cada racha natural) como ya ordenado, y fusiona por pares hacia arriba hasta que quede un todo ordenado.

Timsort combina ambas ideas: identifica las rachas naturales en la entrada, extiende las cortas con ordenamiento por inserción hasta una longitud mínima, luego fusiona las rachas de abajo hacia arriba usando una estrategia de merge cuidadosa que mantiene los tamaños de racha balanceados. El resultado es $O(n)$ sobre entrada ya ordenada — Timsort detecta la única racha grande y no fusiona nada — y $O(n \log n)$ sobre entrada aleatoria, con constantes que se miden bellamente sobre datos del mundo real.

El algoritmo completo de Tim Peters tiene más piezas móviles de las que este capítulo tiene espacio: una pila de rachas con invariantes que gobiernan cuándo fusionar, un *modo galope* que — cuando un lado de un merge sigue ganando — salta hacia adelante en pasos de búsqueda binaria en lugar de avanzar un elemento a la vez, y manejo cuidadoso de rachas descendentes. Implementaré una versión simplificada que captura las dos ideas centrales — ordenamiento por inserción para trozos cortos más fusión de abajo hacia arriba — y me saltaré la optimización de detección de rachas, sobre la que añadiré un párrafo al final.

El paso de merge mismo es el merge del capítulo 4, reorientado para operar sobre un rango del arreglo en lugar de dos listas separadas:

```
/// <summary>
/// Fusiona items[lo..mid] e items[mid..hi] in situ.
/// Ambas mitades deben estar ordenadas.
/// </summary>
private static void MergeRange<T>(
    IList<T> items, int lo, int mid, int hi, Comparison<T> order)
{
    var left = new T[mid - lo];
    var right = new T[hi - mid];
    for (int k = 0; k < left.Length; k++) left[k] = items[lo + k];
    for (int k = 0; k < right.Length; k++) right[k] = items[mid + k];

    int i = 0, j = 0, w = lo;
    while (i < left.Length && j < right.Length)
    {
        if (order(left[i], right[j]) <= 0)
        {
            items[w] = left[i]; i++;
        }
        else
        {
            items[w] = right[j]; j++;
        }
        w++;
    }
    while (i < left.Length) { items[w] = left[i]; i++; w++; }
    while (j < right.Length) { items[w] = right[j]; j++; w++; }
```

```
}
```

Listing 3: `MergeRange`: fusiona `items[lo..mid]` e `items[mid..hi]`

El Timsort simplificado mismo tiene dos fases. Primero, ordena trozos de tamaño fijo (los "min runs") con ordenamiento por inserción. Segundo, fusiona los trozos por pares, duplicando el tamaño de racha cada vez, hasta que quede una sola racha ordenada.

```
/// <summary>
/// Timsort-lite: merge sort de abajo hacia arriba con rachas de
/// ordenamiento por insercion.
/// El Timsort real (Tim Peters, 2002) tambien detecta rachas naturales
/// en la entrada y usa una pila de rachas con invariantes de merge;
/// ambos refinamientos se describen en la prosa del capitulo y se
/// omiten aqui.
/// </summary>
public static void TimSortLite<T>(
    IList<T> items, Comparison<T> order = null, int minRun = 32)
{
    order ??= Comparer<T>.Default.Compare;
    int n = items.Count;
    if (n < 2) return;

    // Fase 1: ordenar trozos de tamano fijo con insercion.
    for (int start = 0; start < n; start += minRun)
    {
        int end = Math.Min(start + minRun, n);
        InsertionSortRange(items, start, end, order);
    }

    // Fase 2: merge de abajo hacia arriba de los trozos ordenados.
    for (int size = minRun; size < n; size *= 2)
    {
        for (int leftStart = 0; leftStart < n; leftStart += 2 * size)
        {
            int mid = Math.Min(leftStart + size, n);
            int rightEnd = Math.Min(leftStart + 2 * size, n);
            if (mid < rightEnd)
                MergeRange(items, leftStart, mid, rightEnd, order);
        }
    }
}
```

Listing 4: Timsort-lite

```
var items = new[] { 3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5, 8, 9, 7, 9, 3, 2,
    3, 8, 4 };
PracticeImpl.TimSortLite(items);
Console.WriteLine(string.Join(", ", items)); // ordenado in situ

// Comprobacion de estabilidad     claves iguales preservan su
// orden relativo
var pairs = new[] { (1, 'a'), (3, 'b'), (1, 'c'), (3, 'd'), (2, 'e'),
    (1, 'f') };
Comparison<(int, char)> byFirst = (x, y) => x.Item1.CompareTo(y.Item1);
PracticeImpl.TimSortLite(pairs, byFirst);
Console.WriteLine(string.Join(", ", pairs));
```

```
1, 1, 2, 2, 3, 3, 3, 3, 4, 4, 5, 5, 5, 6, 7, 8, 8, 9, 9, 9
```

`(1, a), (1, c), (1, f), (2, e), (3, b), (3, d)`

El Timsort real va más allá en una sola dimensión que importa enormemente en la práctica. Antes de la fase 1, recorre la entrada una vez buscando rachas naturales — secuencias ascendentes maximales que ya están en los datos. Cada racha natural se vuelve una unidad; solo las rachas cortas se extienden por ordenamiento por inserción hasta la longitud mínima. La fase de merge entonces opera sobre una mezcla de rachas naturales (potencialmente mucho más largas que `minRun`) y trozos forzados de `minRun`, y la pila de rachas con invariantes de merge mantiene el árbol de merge balanceado. El resultado es adaptativo: la entrada ya ordenada corre en tiempo $O(n)$, la entrada ordenada al revés corre en $O(n)$ (porque las rachas en reversa se detectan y se voltean), y la entrada con aunque sea una pequeña cantidad de estructura corre más rápido que la entrada aleatoria. CPython, OpenJDK y Rust todos envían versiones del algoritmo completo.

4. Las tres preguntas, aplicadas

¿Es correcto?

Ambos algoritmos se reducen a composiciones de algoritmos correctos de capítulos anteriores. Introsort es quicksort con un respaldo de ordenamiento por inserción para subarreglos pequeños. La corrección de quicksort (capítulo 4) cubre el caso recursivo; la corrección del ordenamiento por inserción (capítulo 3) cubre el caso base. El umbral de corte no afecta la corrección — solo el rendimiento. Timsort-lite es ordenamiento por inserción en el primer nivel de trozos pequeños, seguido de fusión estable de abajo hacia arriba. La corrección del ordenamiento por inserción cubre la fase 1; la corrección del merge (capítulo 4) cubre la fase 2; la estabilidad del merge asegura que cada nivel de fusión preserva el orden que estableció el nivel anterior.

La estabilidad separa a los dos. Introsort *no* es estable — quicksort no lo es, y el corte a ordenamiento por inserción no lo restaura — pero Timsort-lite *sí* lo es, porque el ordenamiento por inserción y el merge son ambos estables. El Timsort completo es estable por la misma razón.

¿Cuán eficiente es?

Introsort-lite es $O(n \log n)$ esperado con el pivote aleatorio, $O(n^2)$ en el peor caso improbable (que el introsort completo de Musser mejora a $O(n \log n)$ en el peor caso vía el respaldo de heapsort). El espacio es $O(\log n)$ para la pila de recursión. El corte a ordenamiento por inserción no cambia la complejidad asintótica; mejora el factor constante reduciendo la sobrecarga por recursión en las hojas del árbol de recursión.

Timsort-lite es $O(n \log n)$ en el peor caso y $O(n)$ sobre entrada ya ordenada (porque el primer nivel de fusión encuentra trozos adyacentes ya en orden y el merge no tiene trabajo que hacer más allá del escaneo lineal). El espacio es $O(n)$ — la fase de merge asigna búferes auxiliares proporcionales al tamaño del trozo en cada nivel. El Timsort completo retiene el $O(n \log n)$ del peor caso y mejora el comportamiento del mejor caso sobre entrada parcialmente estructurada mediante el mecanismo de detección de rachas.

En código de producción, el orden asintótico ($O(n \log n)$) te dice qué no cambia cuando las entradas escalan; las constantes te dicen qué sí. Los ordenamientos prácticos ganan atacando las constantes — usando ordenamiento por inserción donde su diminuta constante vence la sobrecarga de merge sort, yendo de abajo hacia arriba para saltarse marcos de recursión, detectando rachas naturales para saltarse trabajo por completo. El análisis asintótico del capítulo 4 no desapareció; se volvió el piso, no el techo.

¿Es óptimo?

Dentro del modelo de comparaciones, ambos algoritmos alcanzan la cota inferior $\Omega(n \log n)$ del capítulo 4. Ninguno es asintóticamente mejor que merge sort o quicksort. En lo que introsort

y Timsort *sí* son óptimos es en rendimiento de reloj real — prestan atención a constantes, comportamiento de caché, y estructura de entrada que el análisis asintótico no vio. Son óptimos en un modelo más rico que el modelo de comparaciones: un modelo que incluye factores constantes, predicción de ramas, y las largas rachas preordenadas que los datos reales tienden a contener.

Teorema 1. *Dentro del modelo de comparaciones, introsort y Timsort cumplen la cota* $\Omega(n \log n)$. *Dentro de un modelo de coste más rico que incluye constantes y estructura de entrada, superan a los ordenamientos teóricos puros por órdenes de magnitud en la práctica.*

Ese es el golpe del capítulo. La cota inferior de comparaciones es una restricción teórica, pero la práctica opera dentro de un modelo de coste mucho más rico de lo que el modelo de comparaciones captura. El ordenamiento de producción gana explotando estructura que el análisis teórico ignoró.

5. Componer, no elegir

Introsort y Timsort son correctos porque son composiciones de algoritmos que ya has verificado. Ambos corren en $O(n \log n)$, con Timsort alcanzando $O(n)$ sobre entradas ya ordenadas. Ambos mejoran el factor constante de los ordenamientos teóricos por órdenes de magnitud en la práctica, usando ordenamiento por inserción donde gana sobre entradas pequeñas y explotando la estructura de los datos del mundo real que la cota inferior teórica no ve.

En código de producción no eliges un algoritmo; compones varios — ordenamiento por inserción para las hojas, quicksort para el divide, merge sort para estabilidad, heapsort como red de seguridad. Cada uno corre solo en el régimen para el que es mejor. La habilidad del practicante es reconocer qué técnica aplica dónde y componerlas sin revolver los invariantes de cada una.

Esta idea — que la optimalidad asintótica es necesaria pero no suficiente — va a volver a lo largo del resto del libro. Las tablas hash de la Parte II usan direccionamiento abierto, encadenamiento, o ambos, dependiendo del factor de carga. Los árboles balanceados de la Parte III usan rotaciones para mantener el peor caso acotado mientras corren rápido en promedio. La búsqueda en grafos de la Parte V usa distintos órdenes de recorrido dependiendo de la pregunta que se hace. En cada caso, el algoritmo teórico es el punto de partida, no el destino. El software real cose los movimientos para las entradas que ve.

El siguiente ensayo corto hace inventario de lo que enseñó la Parte I — la estructura como el ingrediente secreto, el cinosura de las inversiones, la recursión y la aleatorización como armas de diseño, las cotas inferiores pertenecientes a modelos, y el principio de composición que este capítulo acaba de aterrizar.

6. Código completo

```
using System;
using System.Collections.Generic;

namespace Codex.Sort
{
    public static class PracticeImpl
    {
        // ---------- Bloque de construccion: ordenamiento por insercion
            ----------
        private static void InsertionSortRange<T>(
            IList<T> items, int lo, int hi, Comparison<T> order)
        {
            for (int i = lo + 1; i < hi; i++)
            {
```

```
                int j = i;
                while (j > lo && order(items[j], items[j - 1]) < 0)
                {
                    (items[j], items[j - 1]) = (items[j - 1], items[j]);
                    j--;
                }
            }
        }

        // ---------- Introsort-lite ----------
        public static void IntroSort<T>(
            IList<T> items, Comparison<T> order = null,
            int insertionCutoff = 16, Random rng = null)
        {
            order ??= Comparer<T>.Default.Compare;
            rng ??= Random.Shared;
            IntroSortRange(items, 0, items.Count, order, insertionCutoff
                , rng);
        }

        private static void IntroSortRange<T>(
            IList<T> items, int lo, int hi,
            Comparison<T> order, int cutoff, Random rng)
        {
            if (hi - lo <= cutoff)
            {
                InsertionSortRange(items, lo, hi, order);
                return;
            }
            int pivotIdx = rng.Next(lo, hi);
            (items[pivotIdx], items[hi - 1]) = (items[hi - 1], items[
                pivotIdx]);
            int p = QuickSortImpl.Partition(items, lo, hi, order);
            IntroSortRange(items, lo, p, order, cutoff, rng);
            IntroSortRange(items, p + 1, hi, order, cutoff, rng);
        }

        // ---------- Merge sobre rango ----------
        private static void MergeRange<T>(
            IList<T> items, int lo, int mid, int hi, Comparison<T> order
                )
        {
            var left = new T[mid - lo];
            var right = new T[hi - mid];
            for (int k = 0; k < left.Length; k++) left[k] = items[lo + k
                ];
            for (int k = 0; k < right.Length; k++) right[k] = items[mid
                + k];

            int i = 0, j = 0, w = lo;
            while (i < left.Length && j < right.Length)
            {
                if (order(left[i], right[j]) <= 0)
                    items[w++] = left[i++];
                else
                    items[w++] = right[j++];
            }
            while (i < left.Length) items[w++] = left[i++];
```

```
            while (j < right.Length) items[w++] = right[j++];
        }

        // ---------- Timsort-lite ----------
        public static void TimSortLite<T>(
            IList<T> items, Comparison<T> order = null, int minRun = 32)
        {
            order ??= Comparer<T>.Default.Compare;
            int n = items.Count;
            if (n < 2) return;

            for (int start = 0; start < n; start += minRun)
            {
                int end = Math.Min(start + minRun, n);
                InsertionSortRange(items, start, end, order);
            }

            for (int size = minRun; size < n; size *= 2)
            {
                for (int leftStart = 0; leftStart < n; leftStart += 2 *
                    size)
                {
                    int mid = Math.Min(leftStart + size, n);
                    int rightEnd = Math.Min(leftStart + 2 * size, n);
                    if (mid < rightEnd)
                        MergeRange(items, leftStart, mid, rightEnd,
                            order);
                }
            }
        }
    }
}
```

Listing 5: `Practice.cs` — introsort-lite y Timsort-lite

7. Notas y lecturas adicionales

Introsort fue introducido por David Musser en 1997 en "Introspective Sorting and Selection Algorithms", *Software: Practice and Experience* 27(8). El artículo es corto y vale la pena leerlo; el respaldo a heapsort limitado por profundidad es la idea genuinamente original. El `std::sort` de C++ adoptó introsort poco después, y es por eso que los programadores de C++ desde finales de los 90 han tenido una única función de ordenamiento con garantías de peor caso $O(n \log n)$.

Timsort fue desarrollado por Tim Peters para CPython en 2002. Peters escribió un largo documento de diseño que todavía está en el árbol de fuentes de CPython y sigue siendo la mejor referencia primaria. El algoritmo completo — con detección de rachas, invariantes de la pila de merge, y modo galope — también está implementado en OpenJDK (desde Java 7) y la biblioteca estándar de Rust. Timsort fue célebremente el sujeto de un esfuerzo de verificación formal: de Gouw et al. (2015) usaron el demostrador de teoremas KeY para verificar la implementación de Timsort de Java y descubrieron un bug real en el mantenimiento del invariante de la pila de merge, que había estado escondido en código de producción en tres runtimes de lenguajes mayores durante años. El parche de corrección del bug era de 4 líneas.

CLRS no cubre el ordenamiento práctico en profundidad; su foco es la cota inferior teórica. *Algorithms* (4.ª ed.) de Sedgewick y Wayne §2.3 tiene un tratamiento de quicksort con sabor a producción que insinúa introsort. El código fuente de CPython — `Objects/listobject.c` y

`Objects/listsort.txt` — es la referencia canónica para los detalles de Timsort si quieres ver cómo se ve el algoritmo completo en C idiomático.